EvoCUA: Evolving Computer Use Agents via Learning from Scalable Synthetic Experience

Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, Jinrui Ding, Xiandi Ma, Yuchen Xie, Peng Pei, Xunliang Cai, Xipeng Qiu

Paper version: arXiv 2601.15876v2 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *Passive imitation of static datasets struggles to capture causal dynamics in long-horizon tasks.*

EvoCUA is a native **desktop** agent: **screenshots in**, coordinates and keys out, trained on synthetic tasks with executable validators (board pattern K13: checker-first synthesis).

Motivation: three challenges

1. **Verifiable data.** Text-only tasks may be infeasible.
2. **Scalable rollouts.** Tens of thousands of async sandboxes.
3. **Efficient recipe.** Focus on boundary tasks.

Evaluation setup

OSWorld-Verified · Pass@1 · 50 steps

Qwen3-VL-32B-Thinking · Qwen3-VL-8B-Thinking · OpenCUA-72B base

- **Online:** OSWorld-Verified success. Offline grounding and general scores: Table 2.
- **Decontaminated, not held-out:** checked against the benchmark.

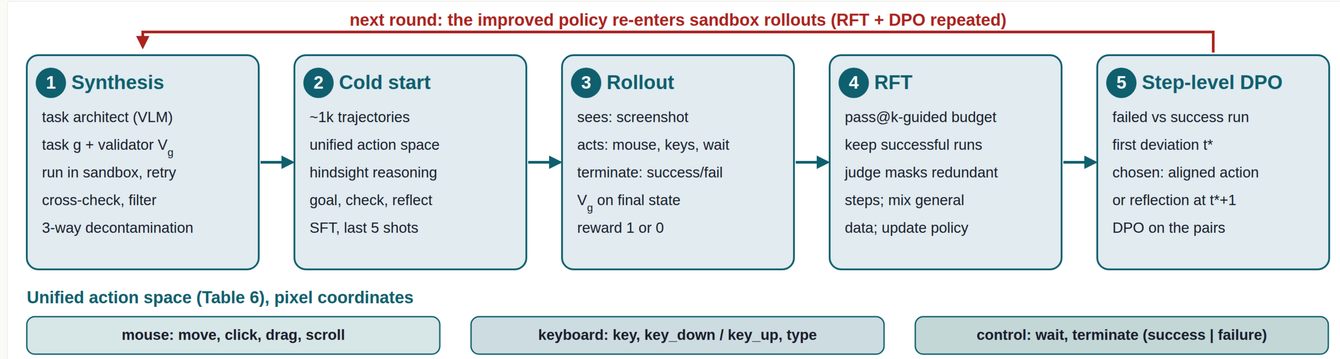
Our read (CUA Radar)

70	★★★	online
score of 100	band	evidence level

- **Reports:** online eval; baselines; cost; ablation; validator spot checks.
- **Flags:** AP4 (judge spot-checked only), AP9. Not reported: variance.
- Pattern K13. Authors' numbers, not rerun.

Method: synthesize, roll out, consolidate, then correct failures

- **Sees / acts:** last 5 screenshots in; mouse, keyboard, wait and terminate out.
- **New:** task and executable validator are co-generated; RFT, then step-level DPO on failures.
- **Signal:** binary validator check on the final state, plus failed-vs-successful run pairs.



Our redraw (not the paper's figure): the evolving cycle of Sections 3 and 5, and the action space of Table 6.

$$\mathcal{J}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(z_w, a_w \mid h_t, o_t)}{\pi_{\text{ref}}(z_w, a_w \mid h_t, o_t)} - \beta \log \frac{\pi_\theta(z_l, a_l \mid h_t, o_t)}{\pi_{\text{ref}}(z_l, a_l \mid h_t, o_t)}\right)\right]$$

Step-level DPO (Eq. 2): chosen (z_w, a_w) vs rejected (z_l, a_l) at the forking step

Key results: OSWorld-Verified success (%)

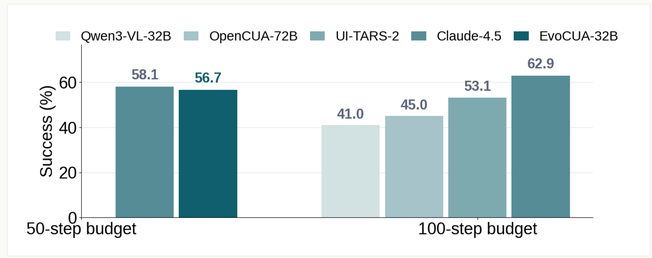


Table 1, redrawn: online, pass@1. Qwen3-VL-32B is the Thinking variant; budgets differ by group.

56.7% in 50 steps; OpenCUA-72B 45.0%, UI-TARS-2 53.1% (100 steps)

Ablation: gain per stage (32B)

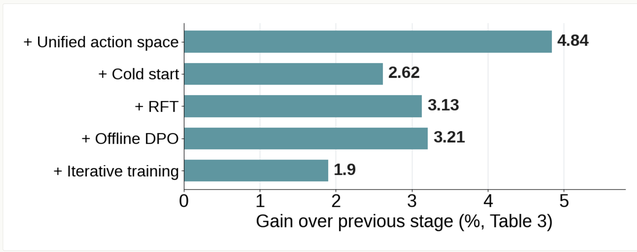


Table 3, redrawn: absolute gain over the previous stage, as reported.

SO WHAT → *Every stage adds a gain; the action space is largest, and a second full RFT + DPO round adds the last one. Table 3 gives no base score, benchmark name or run count. On OpenCUA-72B (Table 4): cold start +2.14, RFT +3.69, DPO +3.02, iterative +1.82. RFT alone, 3 rounds, no cold start: +8.12 (Table 5).*

Headline Numbers

56.7%
OSWorld-Verified success
EvoCUA-32B, 50 steps, pass@1

- **46.1%** EvoCUA-8B, 50 steps
- **45.0%** OpenCUA-72B, 100 steps
- **58.1%** Claude-4.5, 50 steps

The paper's own controls

CONTROL	EVOCUA	COMPARED WITH
Step budget	50 steps	baselines 100 steps
8B, same base	46.1	Step-GUI-8B 40.2
Offline ScreenSpot-Pro	49.76	base 57.10 (*)
Offline MMMU	68.11	base 78.10 (*)

32B offline scores fall; paper cites data mismatch

Takeaway

Checker-first tasks plus RFT and DPO: 56.7% on OSWorld-Verified in 50 steps; offline 32B scores fall.

~1k cold-start traces · 1M+ accelerator hours · no seeds or CIs given

Scan to Read

Paper · Code